

Fair Drop

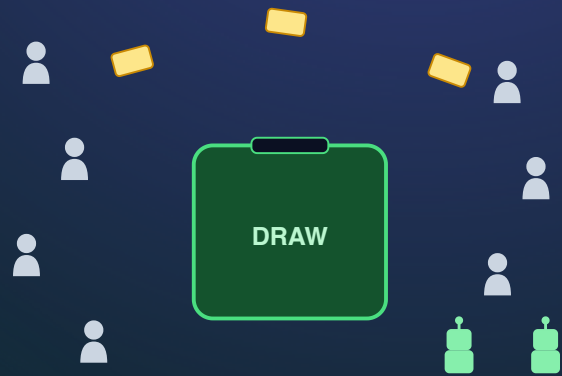
500 seats. 50,000 people. Bots in the crowd.
A ticket sale where being fast doesn't help.

OLD WAY: A RACE



Bots won 100%
of seats, with only 5% of accounts

FAIR DROP: A DRAW



Bots won 4.3%
same attack, fair share

0

seats ever sold twice

0.85×

bot advantage (1.0 = same as a person)

50,000

fake users in every big test

Fair Drop on one page

1 · PROBLEM

In "first click wins", bots win.

Bots click in 0.01 s, people in about 1.5 s.

Bots also spam thousands of requests and never give up.

2 · IDEA

Turn the race into a fair draw.

- 1 Sale stays open for minutes
- 2 Everyone enters once, any time
- 3 Close and lock the list
- 4 Random draw picks winners
- 5 Winners get seats
- 6 Anyone can check it was fair

3 · PROOF (same bots, two systems)

Old way: bots got



Fair Drop: bots got



4 · NO DOUBLE-SELLING

0 seats sold twice. Ever.

The database itself blocks it.

A broken test mode sold 756 of 500,
and our checker caught it.

5 · WHAT WE ADMIT

1,000 fake accounts = 1,000 tickets.

We catch careless cheaters, not careful ones.
Fix: phone check per account.

1 The 60-second pitch

"When 50,000 people chase 500 tickets, it's a race, and bots are faster. We tested it: bots had 5% of accounts and got **100% of seats**. No real person got one.

Fair Drop removes the race. The sale stays open for minutes. Everyone enters **once**, any time. Then a random draw picks winners. Being first doesn't help. Spamming doesn't help, because you only get one entry.

Same bots, our system: they got **4.3% of seats**, matching their 5% share. And no seat was ever sold twice.

We can't cheat the draw either. We lock our secret before the sale and mix in a public random number nobody knows in advance. Anyone can re-check the result.

What we don't fully stop: one person with many fake accounts. We catch careless ones. The real fix is phone verification."

Three things judges should remember



We measured it

100% vs 4.3%. Same attack, 6 runs each.



Never sold twice

Database blocks it. Checker confirms it.



Honest limits

We show the attack we can't stop.

Who answers what

Topic	Sections	Who
Problem, idea, app	1–4	Speaker 1
Test results, numbers	8	Speaker 2
Fair draw, no double-selling, crashes	5, 6	Speaker 3
Bots, fake accounts, limits	7, 9	Keith (builder)

Never guess a number. Say "here's what we measured" and pass it on.

2 The problem and what we built

The ask: a ticket sale where bots can't win by being **faster**, sending **more requests**, or **retrying**. It must survive a huge crowd, never mess up the seat count, and we must **prove it with numbers**.



Bots are faster

0.01 s vs a person's 1.5 s. In a race, bots always win.



Bots spam

Thousands of requests = thousands of chances in a race.



Site crashes

Everyone arrives in the same second.

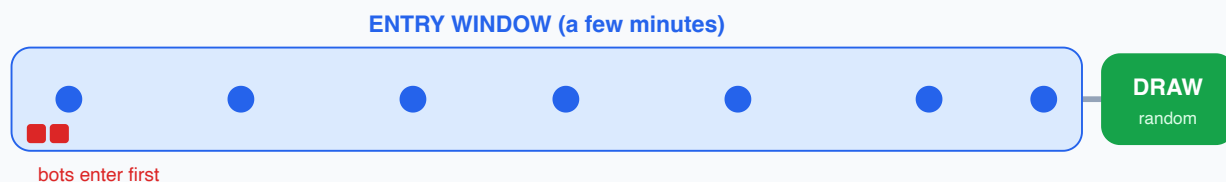
A faster website doesn't fix it. The race just gets tighter.

The 6 required features

Feature	What we built	Proof
Big crowd	One quick database write per entry. Spam blocked early. "Busy, try again" instead of crashing.	50,000 users. Most answered in 3 ms. Everyone answered.
Stop abuse	Random draw, one entry per account, speed limits, spotting fake-account groups.	Flood bots blocked ~17,700 times per test, still one entry each.
No double-selling	Exactly 500 seats in the database, with rules that make double-selling impossible. Separate checker.	0 in every real test. Broken mode: 756 of 500, caught.
Keep user state	Login survives refresh and restart. Refresh mid-click safely re-sends the same request.	Server killed and restarted: users still logged in and entered.
Attack testing	Simulator: 6 user types, 10 scenarios.	36 saved reports.
Measure fairness	Bot vs person win chance; does arriving early help?	Charts in section 8.

3 The big idea: a draw, not a race

When you enter doesn't matter



Everyone gets the same 1% chance ($500 \text{ seats} \div 50,000 \text{ people}$)

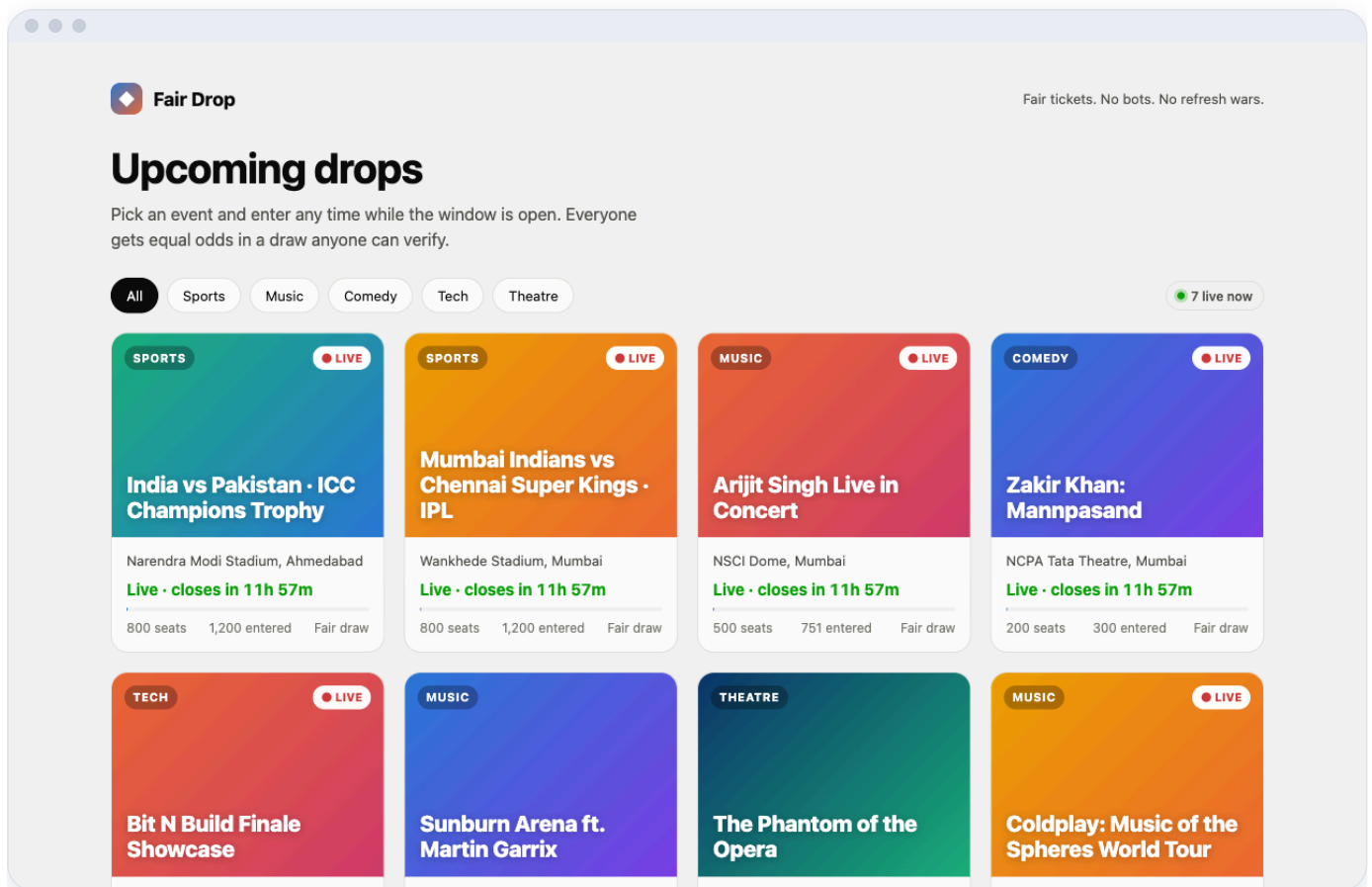
Bot trick	Why it fails now
⚡ Being fastest	The draw ignores entry time.
✉ Spamming requests	One entry per account. Extras get "already in".
🔄 Retrying forever	Same request = same answer. Nothing new.
💥 Flooding the site	Blocked before the database. Window is minutes long, people can retry.
👥 Many accounts	Still works. 1,000 accounts = 1,000 tickets. Our one weak spot.

Smart point: a race gives you a split second to spot bots. A window gives you **minutes** to check all entries calmly before the draw.

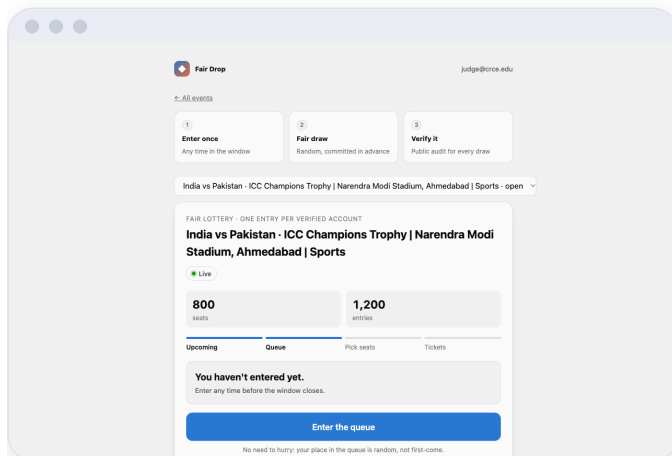
Why not CAPTCHA or a waiting room?

- **CAPTCHA:** bots pay cheap services to solve them; it annoys people; it's still a race after.
- **Waiting room:** place in line is usually by arrival, so bots are still first. A shuffled waiting room *is* a draw, which we built, with proof.

4 The app and how it's built



Event list: seats, entries, "Fair draw". Enter any time while it's open.



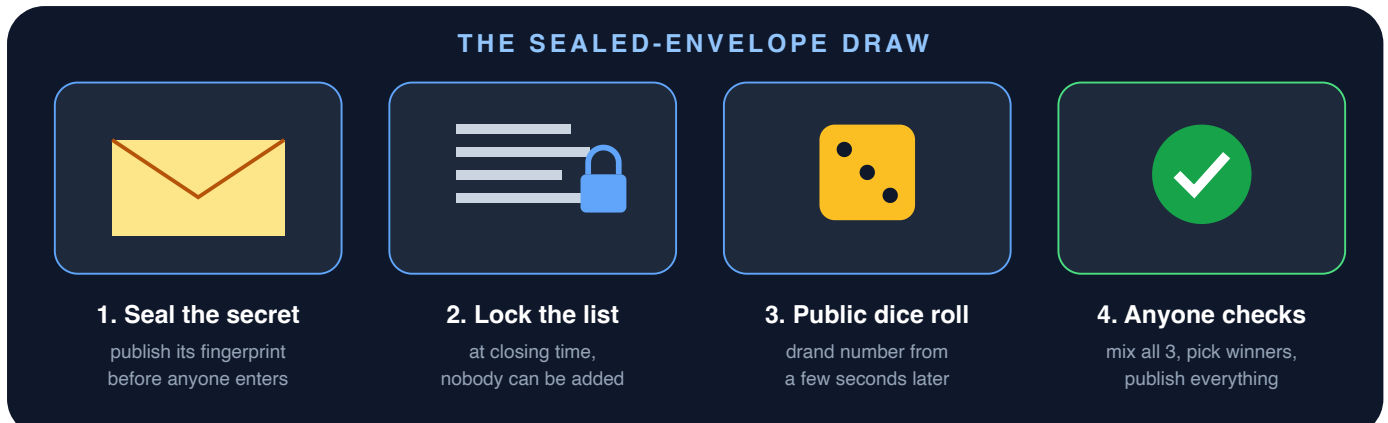
Event page: enter once, fair draw, verify. "No need to hurry."

The parts

- **Next.js** app: website + server in one.
- **PostgreSQL** database: stores everything and enforces the rules.
- **Timer** inside the server: opens, closes, runs the draw.
- **drand**: public random-number service.
- **Simulator**: fakes 50,000 users and bots.
- **Verify script**: anyone can re-check a draw.

No Redis or extra services. One app + one database = less to break.

5 Nobody can cheat the draw (not even us)



1. **Before the sale:** make a secret, publish its fingerprint (hash). Proves we can't change it later.
2. **At close:** lock the entry list, publish its fingerprint too.
3. **Then:** take a **drand** number that comes out a few seconds **after** the lock. Nobody knows it yet, including us.
4. **Draw:** mix secret + list + drand to shuffle everyone. First 500 win.
5. **After:** publish everything. `npm run verify` re-checks it (11 checks, prints DRAW VERIFIED).

Why we can't cheat: to help a friend we'd need the drand number before locking the list. It doesn't exist yet.

drand = free public dice roll run by many independent groups (Cloudflare, universities...). New number every 3 s. No one controls it. We check its digital signature ourselves.

Be honest: we can't prove every account is a real person. If drand is down for 30 s, we draw without it and clearly label it "fallback".

6 No double-selling, and nothing lost on crash

Database rules (can't be broken)

- Exactly 500 seat rows exist.
- One owner per seat. One seat per person.
- One entry per person.
- No entries after closing.
- Draw runs only once (lock).

Proof the checker works: we built a **broken** mode with no safety rules. It sold **756 seats out of 500**. Our checker caught it. Real modes: always 0.

Even plain first-come never oversold. **Correct ≠ fair**. First-come is correct but unfair. Fair Drop is both.

Tricky cases we tested

What if...	Result
One person clicks Enter 50 times at once	1 entry saved. Rest: "already in" or "slow down".
Someone enters at the exact closing second	Close waits for it. Counted, or told "closed". Never half-in.
2,000 buyers race for the seats	Exactly 500 sold, each to one buyer.
Server crashes during the draw	Nothing half-saved. Restart gives the same winners .
Fake drand number sent	Signature fails, never used.

Refresh, bad network, restart

Refresh

Still logged in. A half-sent entry is re-sent safely. Never lost, never doubled.

Bad network

App retries itself. Entering late costs nothing.

Restart

Login is a signed cookie, so it survives. Tested with a hard kill.

Safe retry = every action has a unique ID (idempotency key). Same ID twice → same answer, done once. Like a receipt number.

Not handled: database fully down. Results show within ~3 s (page checks every 3 s).

7 How we handle bots

Defence	Status	Stops	Cost to real people
Random draw over a time window	Built	Fast bots, retriers	Wait a few minutes
One entry per account + safe retries	Built	Spammers, retriers	None
Speed limits per user and per network	Built	Flooders	Crowded Wi-Fi might see "try again"
Spot fake-account groups	Partly	Careless cheaters	0.5% wrongly grouped in tests
Phone verification	Other branch	Fake accounts	Extra sign-up step
Sealed-envelope draw	Built	Insider cheating	None
Secure cookies, CSRF check, unguessable IDs	Built	Tampering	None

Speed limits

- Per user: 2 entries/sec (burst 5). Per network: 50/sec (burst 200), high because a whole college can share one address.
- Database too busy (500+ waiting)? Reply "busy, try again" instead of crashing.
- Bots can't fake their address; the server reads the real connection.

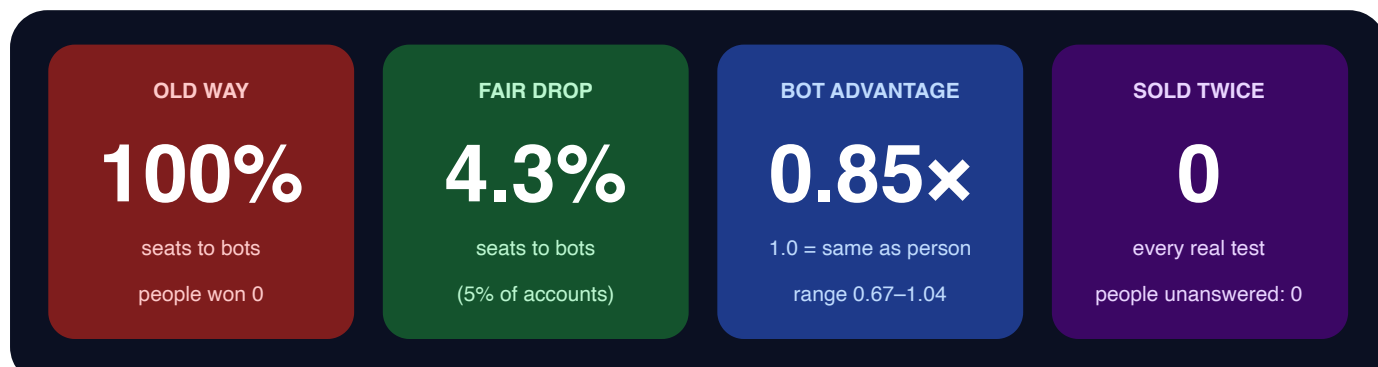
Spotting fake-account groups

At closing, before the draw, we link entries that share a **device**, or are **3+ brand-new accounts** (under 24 h old) from the **same network**. Linked entries count as one. Old accounts on shared Wi-Fi are never linked, so real students are safe. This never reads test labels.

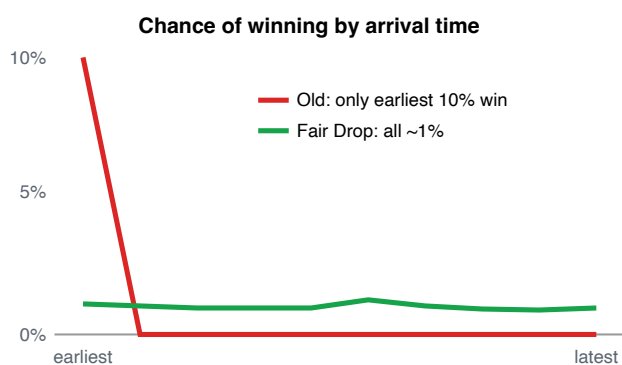
Limit: a careful cheater (different devices, old accounts, spread-out addresses) isn't caught. Real fix: make each account cost something (phone check).

8 Testing and results

A simulator plays 50,000 users against the real server. Main tests repeated 5–6 times. Hidden "bot" labels are only read by the report, never by the defences.



Test	What we threw at it	Bots' seats	Bot advantage
S2	Old first-come + fast bots + flooders	100%	∞ (people won 0)
S3	Fair Drop + same bots	4.3% (3.4–5.2)	0.85x
S4a	2 cheaters × 1,000 accounts, no group-spotting	4.2%	1.05x
S4b	Same, spotting on (careless cheaters)	0% (99.9% caught)	0x
S4c	Same, careful cheaters	4.2% (none caught)	1.05x
S6	Broken mode	756 of 500 sold, caught by checker	
S7 / S8	Retry storm / address hoppers (small tests)	0 / 2 of 50	0x / 0.79x



Arriving early gives nothing in Fair Drop.

Speed in the big crowd

- Half of entries answered in **2–3 ms**.
- Slowest 5%: **~2 s**, only in the opening rush.
- 2–4% got "busy, try again", then worked.
- **Every person got an answer.**
- Old way: people waited ~6 s just to hear "sold out".

Why 0.85, not 1.0?

Luck. Bots expect ~25 seats, ±5 per run. Over 6 runs it ranged 0.67–1.04, around 1.

Also: 67 automated tests on a real database.

9 New seat-picking feature, and our limits

Seat picking (not saved to git yet): venue maps with sections and prices; winners pick up to 6 seats in draw order; all-or-nothing hold for 10 min; pretend payment; unpaid seats return after 5 min. Not part of the test results.

Read before the demo. The current demo events use an "instant queue":

- Each person gets a **random queue number on entry** and is let in ~10 s later. **No shared draw.**
- The "people ahead" number is made up to look real.
- Fake bot entries are added and buy up to half the seats.

It's a **preview**, not the fair system. If asked: *"This demo uses a scripted queue to show seat picking. Our proof is the test results and verified draw."* Safer: demo a normal draw event, show seat picking as the next step.

What we haven't solved (say it first)

Limit	Simple words	Fix later
Fake accounts	Good fakes get their share. Demo accounts are free.	Phone/ID check
Laptop-only load test	50,000 test ran on one machine.	Test on real hosting
One server only	Speed limits live in memory.	Move to Redis
Our own bots	Real bots may be smarter.	Real traffic
Database outage	Sale stops.	Backup database
Not online yet	Deploy ready, not done.	Deploy to Railway

With another week: phone verification · multiple servers + online test · waiting list · live updates · better bot signals.

10 Demo script (~7 min)

```
pg_isready          # database up
npm run build && npm start    # localhost:3000
npm run seed:demo -- --first-open-in 120 --gap 60 --window 600
```

Log in at `localhost:3000/admin/login` with the admin token from `.env`. Use **localhost**, not the IP. Don't restart the server after this.

Step	Show	Say
1. Problem (30 s)	Title	"500 seats, 50,000 people. In a race, bots win."
2. Old way (1 min)	Dashboard S2	"Bots: 5% of accounts, 100% of seats, every time. Seat count was fine; fairness wasn't."
3. Fair Drop (1 min)	S3 + flat chart	"Same bots: 4.3%. Arriving early gives nothing. Everyone got an answer."
4. Fake accounts (1.5 min)	S4a/b/c	"We catch careless cheaters. Careful ones get their share. Fix: phone check."
5. Safety (30 s)	S6	"Broken mode sold 756 of 500. Checker caught it. Real modes: zero."
6. Verify (1 min)	Terminal	<code>npm run verify -- <audit link></code> → "11 passes, DRAW VERIFIED."
7. Live (optional)	Event page	Sign in, enter, refresh (still in), close & draw, see result.

If something breaks

Server down	Show this guide's charts and <code>runs/</code> files. Restart: <code>npm start</code> .
No internet	Draw waits 30 s, runs without drand, says so.
Events closed	<code>npm run seed:demo -- --first-open-in 30 --gap 30 --window 300</code>
Laptop slow	Small run (~70 s): <code>npm run sim -- --scenario S3 --scale 0.1</code>

11 Judge Q&A

Short answer first. More detail only if asked.

Idea

Q1 Why a draw instead of first-come?

First-come really means "fastest bot wins". With 100 people per seat, a draw is the only way a slow phone gets the same chance. Olympic ballots and Ticketmaster Verified Fan use similar ideas.

Q2 Isn't waiting annoying?

A few minutes' wait beats panic-refreshing and losing to bots. Nobody rushes; the site doesn't crash.

Q3 What's new? Lotteries exist.

Anyone can **check** our draw; we **measured** bot impact; the **database** makes double-selling impossible.

Q4 Who would use it?

Concerts, cricket, college fests, product drops, course registration, appointment slots.

Results

Q5 Main result?

Same bots: old way 100% of seats, Fair Drop 4.3%. Zero double-sold.

Q6 What's "bot advantage"?

Bot's win chance ÷ person's. $1.0 = \text{equal}$. Ours: $0.85\times$.

Q7 Why not exactly 1.0?

Luck. ± 5 seats per run moves it ± 0.2 . Range 0.67–1.04 over 6 runs.

Q8 You wrote the bots. Isn't that cheating?

Bot labels are only read by the report, never by the defences. And we show the careful-bot test where we lose.

Q9 Do you block real people?

0.5% were wrongly grouped (test roommates sharing a device). Grouped, not banned. Without group-spotting: 0.

Q10 Did people get through during the attack?

Yes, all of them. Most in 3 ms, slowest ~2 s.

Bots

Q11 Can a bot make 1,000 accounts?

Yes, our one weak spot. Careless ones caught (0% seats). Careful ones get their fair share. Fix: phone verification.

Q12 So you don't stop bots?

We stop what was asked: winning by speed, spam or retrying. Fake identities need identity checks.

Q13 How do you spot fake accounts?

Same device, or 3+ brand-new accounts on one network. Linked = one entry. Shared Wi-Fi with old accounts is safe.

Q14 Device info can be faked.

True. It can only reduce tickets, never add. Faking it beats us (that's test S4c).

Q15 Why no CAPTCHA?

Cheap to solve for bots, annoying for people. Phone checks are stronger.

Q16 1,000 requests a second?

Blocked before the database. Still exactly one entry.

Q17 Changing internet address each time?

Dodges the network limit, but each account still gets one entry. No advantage (S8).

Q18 Can bots crash the site?

Floods are blocked early; window lasts minutes. Everyone got through in tests. Huge network attacks are the host's job.

Q19 Can a bot predict winners?

No. Secret is hidden; drand number doesn't exist until after the lock.

Trust the draw**Q20 Can you rig it?**

Not without getting caught. Sealed secret before, locked list at close, public drand number after. Anyone can verify.

Q21 What's drand?

Public random number every 3 s from many independent groups. Nobody controls it. We check its signature.

Q22 What if drand is down?

Wait 30 s, then draw without it and clearly say so.

Q23 Could you add fake people?

They'd get normal odds and show in the public count. Can't prove every account is real; that's the identity problem.

Q24 Could you unfairly remove people?

We publish how many were removed and why. Not per-person details (privacy), so this is the one thing the check can't fully prove.

Safety and crashes**Q25 How do you stop double-selling?**

Exactly 500 seats; database allows one owner per seat and one seat per person. Checker confirms.

Q26 How do you know the checker works?

Broken mode sold 756 of 500. It caught it.

Q27 Entry at the closing second?

Close waits for it: counted or told "closed".

Q28 Crash mid-draw?

Nothing half-saved; restart gives the same winners.

Q29 Refresh after clicking Enter?

Same request re-sent. Entered once.

Q30 Database goes down?

Sale stops. Not handled yet.

Scale and tech

Q31 Can it handle 50,000?

Yes, on a laptop. Not yet tested on online hosting, and we say so.

Q32 Why no Redis?

The database already keeps things correct. For one day, fewer parts was safer. Redis is the step for multiple servers.

Q33 Why does a draw handle crowds better?

No one rushes, so traffic spreads out. The heavy draw happens once.

Q34 Is it online?

Deploy is ready but not done. Big tests ran locally.

Q35 Why Next.js and Postgres?

Postgres is great at correctness. Next.js = site + API + dashboard in one app.

Q36 Security?

Signed login cookies, CSRF header, admin cookie login, unguessable IDs, encrypted secret.

Q37 Demo login takes any email. Unsafe?

Demo-only, can't take over another email. But yes, accounts are free. Phone check is on another branch.

Tough ones

Q38 Could you break it in 10 minutes?

Yes: 1,000 real-looking accounts (S4c). Everything else holds.

Q39 Everyone enters in the last 5 seconds?

A rush. Spam blocked, busy = "try again". Some might miss out, so the app says "no need to hurry".

Q40 Biggest risk at launch?

Free accounts. Real verification must come first.

Q41 Is the demo queue real?

Normal draw events: yes. Instant-queue events: scripted preview. Say so.

Q42 Who built it?

Keith, solo, in about a day. (*Teammates: be honest about your roles.*)

12 Cheat sheet

Numbers

Seats / people	500 / 50,000 (1% each)
Bots' accounts	5%
Old way: bots' seats	100%
Fair Drop: bots' seats	4.3%
Bot advantage	0.85× (0.67–1.04)
Careless fakes	0% seats, 99.9% caught
Careful fakes	4.2%, none caught
Wrongly grouped	0.5%
Broken mode	756 of 500
Sold twice (real)	0
Answer time	2–3 ms (slowest ~2 s)
People unanswered	0
Flood blocks / test	~17,700
Tests / reports	67 / 36

Words

Hash / fingerprint	Short code from data; can't reverse
drand	Public random number, every 3 s
Idempotency key	Action ID so doing it twice counts once
Rate limit	Speed limit ("slow down", 429)
Sybil attack	One person, many accounts
Transaction	All changes happen, or none
Lock	"Wait your turn" in the database

Commands

```
npm run build && npm start
npm run seed:demo
npm run sim -- --scenario S3
npm run verify -- <audit link>
npm test
```

In the room: open with "100% vs 4.3%". Mention fake accounts before they ask. Never make up a number. Be clear what in the demo is scripted.